AARC: Automated Affinity-aware Resource Configuration for Serverless Workflows

Lingxiao Jin[1*], Zinuo Cai[1*], Zebin Chen[1], Hongyu Zhao[1], Ruhui Ma[1]
[1]School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China
{jinlingxiao1122, kingczn1314, czb453874483, sjtu-zhy, ruhuima}@sjtu.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132894

***Abstract*—Serverless computing is increasingly adopted for its ability to manage complex, event-driven workloads without the need for infrastructure provisioning. However, traditional resource allocation in serverless platforms couples CPU and memory, which may not be optimal for all functions. Existing decoupling approaches, while offering some flexibility, are not designed to handle the vast configuration space and complexity of serverless workflows. In this paper, we propose `AARC`, an innovative, automated framework that decouples CPU and memory resources to provide more flexible and efficient provisioning for serverless workloads. `AARC` is composed of two key components: Graph-Centric Scheduler, which identifies critical paths in workflows, and Priority Configurator, which applies priority scheduling techniques to optimize resource allocation. Our experimental evaluation demonstrates that `AARC` achieves substantial improvements over state-of-the-art methods, with total search time reductions of 85.8% and 89.6%, and cost savings of 49.6% and 61.7%, respectively, while maintaining SLO compliance.**

***Index Terms*—serverless computing, resource configuration, automatization.**

I. Introduction

Serverless computing [1], [2] is emerging as a new paradigm in cloud computing, offering distinct advantages over traditional models like Infrastructure-as-a-Service [3], [4] and Platform-as-a-Service [5], [6]. Unlike these earlier services, serverless computing, or Function-as-a-Service (FaaS), breaks down applications into smaller, more manageable functions. This approach significantly reduces the maintenance burden on developers and enables service providers to maximize the use of underlying hardware resources. Complex applications often require more than one function, which is where serverless workflows come into play [7]. Visualized as directed acyclic graphs (DAGs), serverless workflows can scale based on demand, providing both operational flexibility and cost-effectiveness. Each function within the workflow operates independently, allowing for efficient resource allocation and management.

Despite its reputation for flexible and granular resource allocation, FaaS is still predicated on user-defined quotas for resource configuration. We characterize existing resource management strategies within FaaS frameworks into three primary categories. First, memory-centric configurations in AWS Lambda[1], allow developers to set memory quotas, with vCPU and network bandwidth allocated in proportion to the memory allocation. Second, Google Cloud Functions[2] providers offer a selection of predefined resource combinations. Third, there are flexible yet limited configurations, like Alibaba Cloud Function[3], which enforces a memory-to-CPU ratio within a specified range. It is speculated that these constraints aid cloud providers in the efficient management of resources and potentially reduce the need for extensive physical infrastructure.

However, the existing coupled resource allocation mechanism may not be optimal for all serverless functions. Bilal *et al.* [8] highlight the benefits of resource decoupling, which can potentially reduce execution costs by up to 40% when compared to coupled configurations. However, their study is limited to individual functions and does not extend to workflows. We analyze three serverless workflows [9] with decoupled CPU-memory resources to evaluate their performance in §II-A. Our findings indicate that a coupled CPU-memory mechanism can result in resource inefficiency and increased costs due to the varying resource requirements of different workflows. For instance, in the ML Pipeline workflow, a decoupled configuration of 4 vCPUs and 512 MB memory achieves optimal cost by reducing memory usage by 87.5%, substantially decreasing the overall cost compared to the coupled approach.

While decoupling CPU and memory seems a promising strategy for adaptive resource provisioning, it significantly broadens the configuration space, presenting substantial challenges in finding optimal settings. Previous research [8] suggests a Bayesian optimization-based method for resource allocation, yet this method is geared towards individual functions. We extend the method to workflows and evaluate it with a Chatbot application in §II-B. After 100 rounds of sampling, with the algorithm still not converging, we observe a 32.13% reduction in cost, but the total runtime extends to 9.76 hours, indicating a struggle to reach an optimal solution. Moreover, the cost exhibits frequent fluctuations and nearly half of these changes are increases. These challenges have spurred our efforts to develop a more automated, stable, and efficient resource configuration scheme for the decoupling of resources

This work was supported by Shanghai Key Laboratory of Scalable Computing and Systems. Lingxiao Jin and Zinuo Cai equally contributed to this work. (Corresponding author: Ruhui Ma).

[1]https://aws.amazon.com/lambda/pricing/
[2]https://cloud.google.com/functions/pricing-1stgen
[3]https://www.aliyun.com/product/fc/

and workflows.

In this paper, we introduce AARC, an automated framework designed to configure resources with affinity awareness for serverless workflows. Our key insight lies in the decoupling of computation and storage resources, which enhances flexibility in provisioning resources for serverless workloads. AARC offers three distinct advantages over existing methods. Firstly, AARC surpasses memory-centric allocation schemes [9]–[11] by decoupling memory and CPU, thereby increasing resource flexibility and efficiency through a comprehensive exploration of serverless workflows' resource affinities. Secondly, in contrast to peer works utilizing Bayesian optimization for decoupled configurations [8], our method efficiently manages complex workflows and minimizes the sampling iterations required. Lastly, AARC's resource allocation centers on Service Level Objectives (SLOs), which specify end-to-end latency limits, eliminating manual resource allocation by developers and automating the process.

Specifically, AARC integrates Graph-Centric Scheduler and Priority Configurator to optimize serverless workflow costs using a heuristic algorithm. The Graph-Centric Scheduler starts by breaking down the workflow to find the critical path, which the Priority Configurator then schedules with priority techniques. This is followed by Graph-Centric Scheduler setting sub-SLOs for related sub-paths and determining resource configurations for functions within them. Cloud vendors can use AARC to find optimal resource configurations that meet end-to-end SLOs upon receiving workflows from developers. Our experiment illustrates that during searching for the optimal configuration, AARC reduces the total runtime by around 85% compared to SOTA methods. When comparing the optimal configurations, AARC achieves cost savings of around 50% compared to SOTA methods, while still meeting the SLO requirements.

Our contributions are highlighted as follows.

- Firstly, we identify that memory-centric resource configuration schemes on mainstream serverless computing platforms are not a one-size-fits-all approach, and existing methods for decoupled resource configuration don't work for workflows because of the larger search space.
- Secondly, we propose AARC, a framework for decoupled resource configuration in serverless workflows automatically. Leveraging critical path decomposition and priority scheduling, the framework provides a cost-efficient resource allocation strategy with SLO compliance.
- Finally, our experiment demonstrates that AARC reduces total runtime by 85% and achieves 50% cost savings compared to SOTA methods while meeting SLOs.

II. Motivation and Challenge

A. *Motivation*: Decoupling for Flexibility

While memory-centric resource configurations like AWS Lambda simplify resource management, they may not suit all serverless workloads. Bilal *et al.* [8] demonstrate that resource decoupling can reduce execution costs by up to 40% compared

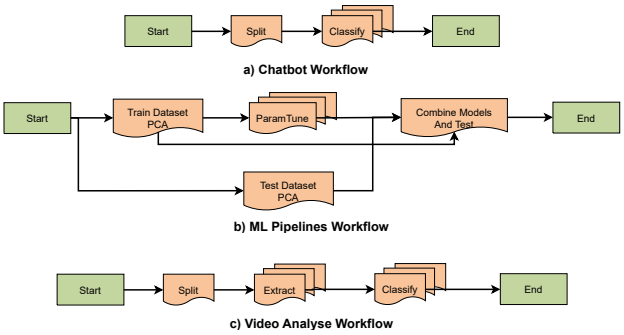


Fig. 1: Architecture of Three Workflows.

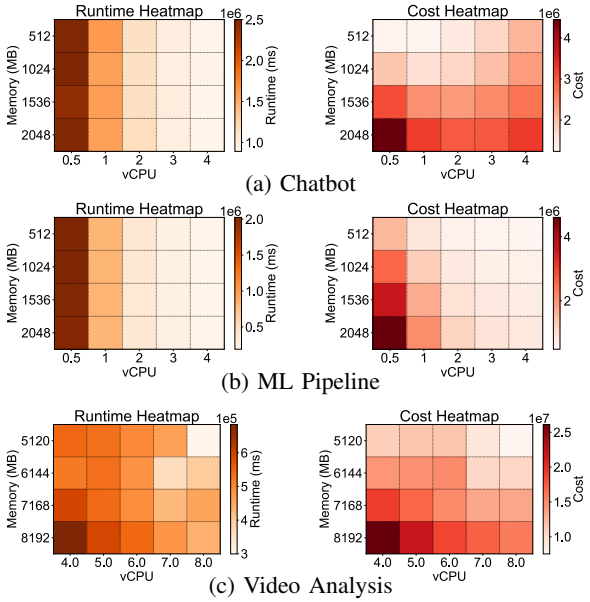


Fig. 2: Runtime and Cost with Decoupled Resources.

to coupled configurations. However, their analysis is restricted to individual functions and does not consider workflows. To explore the impact of decoupled CPU-memory resources for workflows on runtime and cost, we conduct experiments with three different workflows [9], Chatbot, ML Pipeline, and Video Analysis. Fig. 1 illustrates the architecture of these workflows. The Chatbot workflow processes input, trains classifiers in parallel, and uses remote storage for real-time intent detection. The ML Pipeline workflow achieves machine learning by performing dimensionality reduction, model training, and testing. The Video Analysis workflow splits input videos, extracts key frames, and classifies them.

Fig. 2 demonstrates the impact of decoupling CPU and memory on the runtime and cost of the workflows. Fig. 2a and Fig. 2b show that the runtime for Chatbot and ML Pipeline remains unchanged despite memory variations, indicating inefficiency in memory-centric resource allocation for computation-intensive tasks. Notably, in the ML Pipeline workflow, a decoupled configuration of 4 vCPUs and 512 MB memory achieves the lowest cost, reducing memory usage by 87.5% compared to the coupled approach, underscoring the need for decoupled strategies. Additionally, comparing Fig. 2a and Fig. 2c reveals distinct resource affinities across

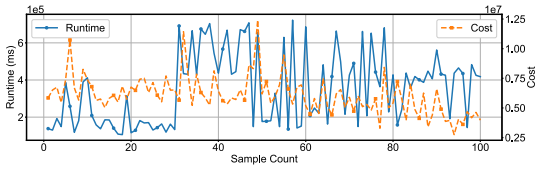


Fig. 3: Bayesian Optimization Search for Chatbot.

workflows. For example, Chatbot minimizes costs with 512 MB memory and 1 vCPU, while Video Analysis achieves cost efficiency with 5120 MB memory and 8 vCPUs. Thus, resource configurations should be tailored to each workflow to minimize costs while meeting SLOs.

B. *Challenge*: Larger Configuration Space after Decoupling

Decoupling CPU and memory improves resource utilization and reduces costs but significantly expands the configuration space, complicating optimal resource provisioning. Profiling-based methods [9], [12], [13] face increased sampling challenges as configurable resources grow. The complexity of serverless applications is further exacerbated by the fact that 46% of applications involve multiple functions [7]. While prior work [8] explores Bayesian optimization for decoupled resource configuration, it focuses on individual functions, overlooking the unique demands of workflows.

We extend the Bayesian Optimization (BO) method from this study to serverless workflows, specifically testing the Chatbot application, to observe changes in runtime and cost with increased sampling. As depicted in Fig. 3, while the cost decreases by 32.13% over 100 sampling rounds without converging, the total runtime extends to 9.76 hours. Additionally, the cost experiences frequent fluctuations, with over half of the changes being increases. The average fluctuation amplitude, calculated as the mean absolute difference between consecutive values, accounts for 18.3% of the mean, which highlights its significant instability. This instability stems from the expansion of the search space when applying decoupling to workflows, which complicates the identification of the optimal solution using Bayesian Optimization. Consequently, current methods struggle to optimize serverless workflows effectively.

III. Design

A. Overview

Fig. 4 depicts the overall architecture of AARC consisting of two main components: Graph-Centric Scheduler and Priority Configurator. Graph-Centric Scheduler decomposes the input workflow and identifies its critical path, along with all the sub-paths linked to it. Priority Configurator configures each function along the critical path and sub-paths.

In our proposed framework, developers submit ❶ their workflow to the cloud platform along with the SLO. First, the Graph-Centric Scheduler profiles the user-defined workflow based on dummy input, calculating the runtime for each function in the workflow. This runtime is then used as the weight of each node, converting ❷ the workflow into a weighted DAG. Next, the Scheduler extracts the critical path and its SLO from the weighted DAG and passes ❸ this information to the

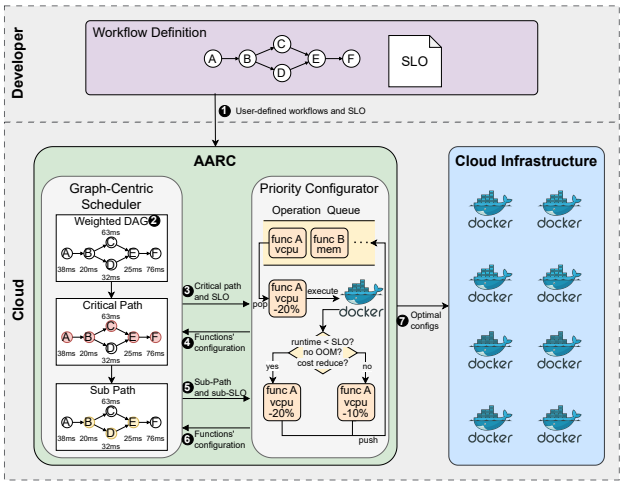


Fig. 4: System Architecture.

TABLE I: Functions in Algorithm 1 and Algorithm 2.

Functions	Description
deallocate(op)	According to the type and step of the given operation op, deprive a portion of the function's resources and return the runtime and cost under this configuration.
allocate(op)	According to type and step of the given operation op, allocate a portion of resources to the function and return a new step and trail of the op.
find_critical_path(G)	Given a weighted DAG, return the critical path of this DAG.
find_detour_subpath(G, critical_path)	Given a DAG and its critical path, return all the sub-paths connected to the critical path.
runtime_sum(path, start, end)	Given the start and the end of a path, calculate the overall duration time between the start and the end.

Priority Configurator. Priority Configurator then incrementally reduces the memory and CPU allocation for each function through priority-based scheduling to determine ❹ the optimal resource configuration that meets the SLO. Based on the optimal configuration of the critical path and the SLO, Graph-Centric Scheduler substitutes functions on the non-critical path onto the critical path to generate ❺ sub-paths and corresponding SLOs. Similarly, Priority Configurator calculates ❻ the optimal resource configuration of the functions in sub-paths, ensuring that the critical path's SLO is not violated. Finally, Graph-Centric Scheduler finalizes ❼ the resource configuration for subsequent container resource allocation, which ensures SLO compliance with optimal cost efficiency.

B. Detailed Design

Graph-Centric Scheduler employs *Overall Scheduling Algorithm* to process the workflow and SLO, performing critical path-based decomposition of the workflow. It then invokes the Priority Configurator to allocate resources for functions along the paths. Priority Configurator uses *Priority Configuration Algorithm* to handle the function paths and SLO, achieving decoupled resource allocation through priority scheduling. The pseudocode for the *Overall Scheduling Algorithm* and

Algorithm 1: Overall Scheduling

Input: function workflow G, end-to-end latency SLO
Output: configurations for each function G_configs

1 **Function** ***schedule***(G, SLO)**:**
2 /* assign base configuration */
3 **foreach** $v_i \in$ G **do**
4 v_i.config ← base_config
5 **end**
6 /* execute to find critical path */
7 execute G
8 L ← ***find_critical_path***(G)
9 G_configs ← {}
10 configs ← ***priority_configuration***(L, SLO)
11 G_configs ← G_configs ∪ configs
12 /* compute configs for subpaths */
13 subpaths ← ***find_detour_subpath***(G, L)
14 **foreach** sp ∈ subpaths **do**
15 SLO′ ← ***runtime_sum***(L, sp.start, sp.end)
16 **foreach** $v_i \in$ sp **do**
17 **if** v_i is scheduled **then**
18 v_i ← ***pop***(sp)
19 SLO′ ← SLO′ − v_i.runtime
20 **end**
21 **end**
22 configs ← ***priority_configuration***(sp, SLO′)
23 G_configs ← G_configs ∪ configs
24 **end**
25 **return** G_configs
26 **End Function**

Algorithm 2: Priority Configuration

Input: function path L, end-to-end latency SLO
Output: configuration for each function configs

1 **Function** ***priority_configuration***(L, SLO)**:**
2 PQ, count ← ***priority_queue***(), 0
3 **foreach** $v_i \in L$ **do**
4 **for** type ∈ [cpu, mem] **do**
5 func, priority ← v_i, ∞
6 step, trial ← 1, FUNC_TRIAL
7 op ← {func, type, step, trail}
8 ***push***(PQ, op, priority)
9 **end**
10 **end**
11 **while** ***len***(PQ) > 0 **and** count $<$ MAX_TRAIL **do**
12 op, count ← ***pop***(PQ), count $+ 1$
13 runtime, cost ←***deallocate***(op)
14 **if** runtime $>$ SLO **or** cost increases **then**
15 op.trail, op.step ←***allocate***(op)
16 **if** op.trail > 0 **then**
17 ***push***(PQ, op, 0)
18 **end**
19 **else**
20 priority ← reduced cost
21 ***push***(PQ, op, priority)
22 **end**
23 **end**
24 configs ← $\{(v_i.cpu, v_i.mem) \mid v_i \in L\}$
25 **return** configs
26 **End Function**

Priority Configuration Algorithm is provided in Algorithm 1 and Algorithm 2, with their functions explained in TABLE I.

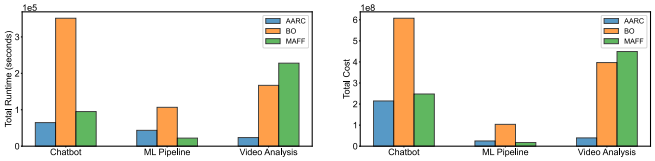
Algorithm 1 presents the whole scheduling process. Given a function workflow and the target end-to-end SLO, the algorithm operates as follows: Firstly, each function in the workflow is initially assigned an over-provisioned base configuration to ensure the workflow meets the SLO in most cases (Line 2-4). The workflow is then executed, and the runtime for each function is used as the weight in a DAG to identify the critical path (Line 5-6). The critical path and the end-to-end SLO are then utilized as inputs for the Priority Configurator, which in turn generates optimized configurations for each function along the critical path (Line 7-9). Through a full DAG traversal, the algorithm identifies sub-paths linked to the critical path, defined by their start and end nodes within the critical path, and no intersections with other nodes (Line 10). For each sub-path, the algorithm calculates the sub-SLO as the time interval between critical path nodes, ensuring critical path consistency during scheduling (Line 12). The Priority Configurator configures each function along the sub-path, following the same method as with the critical path (Line 19-20). To prevent conflicts and ensure each function only schedules once, the algorithm sets a scheduled flag for every function. After scheduling a function, the algorithm removes it from the path and adjusts the SLO accordingly (Line 13-18).

Algorithm 2 presents *Priority Configuration Algorithm*, which adopts priority scheduling to optimize the configuration of sequentially executed functions subject to SLO limitation. The algorithm initializes a priority queue PQ to manage operations for each function (Line 2), which requires two distinct operations to adjust CPU and memory quotas (Line 3-10). Based on the operation type, the algorithm modifies the function's resources, continuously executing operations from PQ and monitoring their impact on runtime and cost (Line 12-13). If an operation violates the SLO, increases cost, or encounters an error, the algorithm reverts the resource (Line 14-18). Simultaneously, an exponential backoff mechanism reduces the step size to ensure convergence while avoiding over-scheduling (Line 15). Operations consistently causing violations (trail $= 0$) are removed from PQ (Line 16), while those with potential are re-enqueued with adjusted priorities (Line 17, 20-21). The loop terminates when PQ is empty or a user-defined iteration limit, $\text{MAX}_{\text{TRAIL}}$, is reached (Line 11).

IV. Performance Evaluation

A. Experiment Setup

a) Environment: The experiments run on a machine with 4-socket Intel Xeon Gold 6248R (96 physical core),



(a) Total Runtime (b) Total Cost

Fig. 5: Overall Sample Cost and Runtime Comparison.

TABLE II: Average Runtime and Cost Comparison.

	Chatbot		ML Pipeline		Video Analysis	
	Runtime (s)	Cost	Runtime (s)	Cost	Runtime (s)	Cost
AARC	103.7±3.2	2390.9k	77.1±2.6	435.0k	316.8±6.6	53.6k
BO	114.7±1.9	4275.2k	60.0±0.7	863.5k	519.9±8.3	82.4k
MAFF	115.3±3.1	3477.5k	109.5±2.0	1136.6k	578.2±19.3	98.8k

and 512GB memory. Workflows execute in separate Docker containers, enabling CPU and memory allocation decoupling.

b) Baselines: We compare our approach with two baselines: Bayesian Optimization (BO) [8] and MAFF gradient descent [14]. Originally designed for single-function resource configuration and memory-centric tasks, these methods have been adapted for workflow optimization. In the Bayesian Optimization method, configuration parameters are discretized to limit the search space. Memory allocation is available in 64 MB increments from 128 MB to 10,240 MB, while vCPU cores range from 0.1 to 10, independently of memory. The MAFF gradient descent method iteratively minimizes cost, allocating vCPU cores proportionally (1 core per 1,024 MB of memory). If a workflow's SLO is violated, the process reverts to the previous step and terminates.

c) Workloads: Our experiments utilize three workflows (Chatbot, ML Pipeline, and Video Analysis), with SLOs set to 120s, 120s, and 600s, respectively. These applications capture key characteristics of serverless DAGs. Specifically, Video Analysis and Chatbot exhibit a scatter communication pattern, while ML Pipeline follows a broadcast pattern [9].

d) Metrics: We evaluate performance using runtime and cost metrics, extending the pricing model of AWS Lambda to decoupled resources. Let $\texttt{cost}_{ij}$ represent the cost of serverless function $\texttt{v}_i$ configured to $(\texttt{cpu}_j, \texttt{mem}_j)$ with runtime $\texttt{t}_{ij}$. We denote the price per vCPU-second as μ_0, the price per GB-second as μ_1, and the price for function requests and orchestration as μ_2. All are constants. The cost equation is then $\texttt{cost}_{ij} = \texttt{t}_{ij}(\mu_0 \cdot \texttt{cpu}_j + \mu_1 \cdot \texttt{mem}_j) + \mu_2$. Here, μ_0, μ_1, and μ_2 are set to 0.512, 0.001, and 0, respectively.

B. Effectiveness of Configuration Search

In the configuration search experiment, we set the initial configurations for each workflow. Then, we perform the configuration search using three methods: AARC, Bayesian Optimization, and MAFF.

a) Overall Efficiency: Fig. 5 shows the total runtime and cost of the sampling process. In all workflows, AARC outperforms the Bayesian Optimization, especially in the Video Analysis workflow, reducing runtime by 85.8% and cost by 90.1%. This improvement stems from AARC's priority scheduling strategy for decoupled resources, which expands the configuration search space while requiring fewer samples. In the Chatbot workflow, AARC performs 64 samples, slightly more than MAFF's 61, but achieves a 31.9% reduction in runtime and 13.4% in cost. This is because MAFF's proportional allocation scheme reduces the search space but risks local optima, leading to higher runtime and costs due to resource coupling. In the Video Analysis workflow, AARC reduces runtime and cost by 89.6% and 91.3% compared to MAFF. However, in the ML Pipeline workflow, AARC samples 50 times compared to MAFF's 15, resulting in lower runtime and cost for MAFF. This is due to the ML Pipeline's high CPU and low memory demands, where proportional adjustments often hit local optima, requiring fewer samples to meet exit conditions.

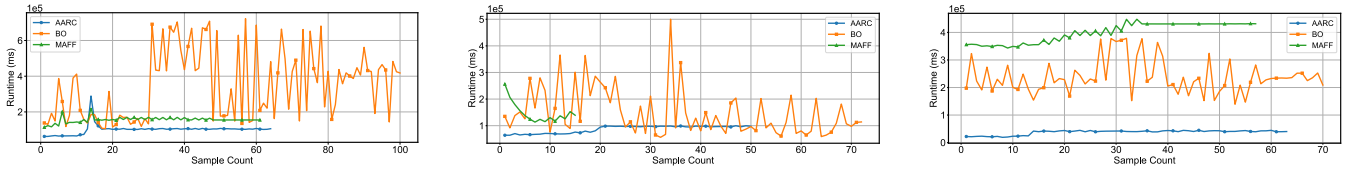
b) Sampling Efficiency: We further illustrate the effectiveness of optimal configurations calculated by each method as the sampling count increases, evaluating these outcomes through workflow runtime and cost. Fig. 6 shows the change in runtime with sampling count under different configurations for each workflow. Our goal is to minimize cost while meeting the SLO, so runtime shows an upward trend using AARC. The Bayesian Optimization method has the highest sampling count and significant instability, as its efficiency decreases due to the large search space created by resource decoupling. In contrast, our priority scheduling strategy maintains search efficiency while accelerating convergence. Fig. 7 depicts the change in cost with sampling count under different configurations. Using AARC, cost shows a downward trend and converges with fewer samples. In the ML Pipeline workflow, the MAFF method quickly falls into local optima due to its coupled resource configuration search approach, making it difficult to discover more cost-effective configurations.

C. Performance of Optimal Configuration

To validate the performance of AARC in meeting the SLO requirements and execution cost, we execute the workflow 100 times using the configurations generated by the aforementioned methods and calculate its average runtime and cost.

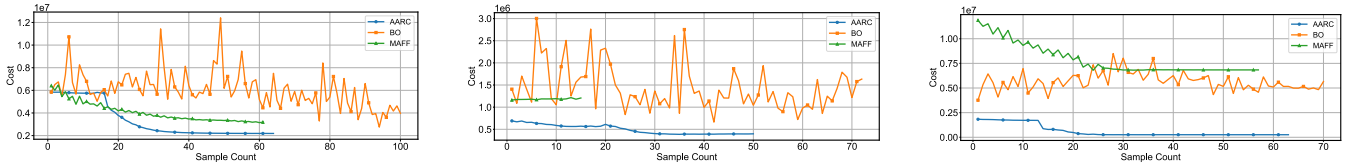
a) SLO Violation: All methods meet the SLO constraints, with the average runtime and standard deviation shown in TABLE II. AARC satisfies SLO because the algorithm reverts resources during configuration search when SLO violations occur and incorporates sub-paths into the critical path, ensuring both critical path consistency and SLO compliance. This demonstrates the effectiveness of our approach in meeting SLO requirements and its potential for integration into real-world systems with high reliability and performance standards.

b) Cost Reduction: TABLE II shows the execution cost of workflows with configurations found by different methods. In the Chatbot, ML Pipeline, and Video Analysis workflows, AARC reduces costs significantly compared to Bayesian Optimization and MAFF. The reductions are 44.0% and 31.2% for Chatbot, 49.6% and 61.7% for ML Pipeline, and 34.9% and 45.7% for Video Analysis, respectively. Compared to the Bayesian Optimization method, our approach uses priority scheduling, which allows for more stable identification of suitable decoupled resource configurations. Unlike MAFF's



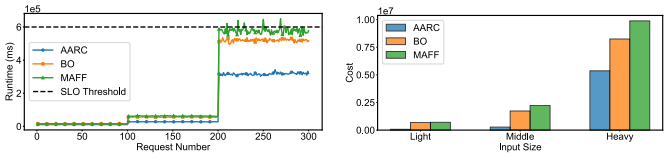
(a) Chatbot (b) ML Pipeline (c) Video Analysis

Fig. 6: Runtime Changing with Sample Counts of Different Methods under Different Workflows.



(a) Chatbot (b) ML Pipeline (c) Video Analysis

Fig. 7: Cost Changing with Sample Counts of Different Methods under Different Workflows.



(a) Runtime (b) Cost

Fig. 8: Performance Across Input Sizes in Video Analysis.

proportional configuration method, our approach achieves decoupled resource allocation, resulting in lower-cost configurations. Since the ML Pipeline has high CPU demands and low memory demands, decoupling resources yields better results.

D. ***Discussion***: *Input-Aware Configuration*

Considering that workflow execution efficiency can be input-sensitive, we further enhanced our design by adding an Input-Aware Configuration Engine Plugin. The Video Analysis workflow is input-sensitive, where different input video sizes correspond to different optimal configurations. If developers trigger the plugin, the Engine analyzes the characteristics of the input data, such as video bitrate and duration. Based on the identified features, the Engine sorts the inputs and invokes Graph-Centric Scheduler and Priority Configurator to determine the optimal resource configuration scheme for each input. When a request arrives, the Engine analyzes the input scale and allocates the input to different configurations.

To validate our design, we evaulate Video Analysis workflow using three input sizes: light, middle, and heavy. Fig. 8a shows workflow runtime with light, middle, and heavy inputs in sequence. Since MAFF does not adjust configurations for input-sensitive workflows, it may violate the SLO under heavy inputs, while our method consistently remains within SLO limits. Fig. 8b shows the average cost of the workflow under the three input sizes. Because our method can select configurations based on input size, it optimizes cost by 89.9% and 89.8% compared to MAFF and Bayesian Optimization under light input, and by 45.7% and 34.9% under heavy input.

V. Related Work

a) Serverless Resource Configuration: Configuring resources in serverless computing involves optimizing memory and CPU allocations to balance performance and cost. Offline approaches like MAFF [14] and COSE [12] rely on optimization algorithms and performance modeling to determine configurations. Bilal *et al.* [8] explores decoupling memory and CPU configurations, leveraging diverse VM types to enhance optimization but still faces challenges with workload variability. To address such dynamism, online configuration methods have emerged. Sizeless [15] uses real-time performance data and prediction models to adjust resources dynamically. FaaSDeliver [16] further refines this by monitoring per-invocation metrics and employing advanced estimators for resource allocation. However, these methods introduce overhead from continuous monitoring and data processing, potentially inflating costs for serverless applications.

b) Serverless Workflows Optimization: Due to the prevalence of workflows in serverless workloads, workflow configuration optimization has gained attention from academia and industry [17]–[19]. SLAM [20] uses distributed tracing to detect relationships between functions and leverages execution times under different memory configurations to estimate overall application execution time. Raza *et al.* [21] improve COSE for serverless workflows, optimizing performance and cost across the entire workflow. StepConf [13] provides dynamic memory allocation for serverless workflows, considering intra-function and inter-function parallelism to fully utilize the parallel capacity of serverless functions. However, these methods still do not effectively decouple resources, leaving room for further optimization.

VI. Conclusion

We propose AARC, an affinity-aware resource configuration framework for serverless workflows, capitalizing on the concept of resource decoupling within the serverless paradigm. By leveraging our framework, developers are relieved of the burden of manual serverless workflow configuration. Experimental results indicate that, during the process of searching for the optimal configuration, AARC achieves a total runtime reduction of 85.8% and 89.6% compared to Bayesian Optimization and MAFF, respectively. Additionally, it reduces resource usage by 49.6% and 61.7% while satisfying the SLO requirements.

REFERENCES

[1] Z. Li, L. Guo, J. Cheng, Q. Chen, B. He, and M. Guo, "The serverless computing survey: A technical primer for design architecture," *ACM Computing Surveys*, vol. 54, no. 10s, pp. 1–34, 2022.

[2] A. Mampage, S. Karunasekera, and R. Buyya, "A holistic view on resource management in serverless computing environments: Taxonomy and future directions," *ACM Computing Surveys*, vol. 54, no. 11s, pp. 1–36, 2022.

[3] S. S. Manvi and G. K. Shyam, "Resource management for infrastructure as a service (IaaS) in cloud computing: A survey," *Journal of Network and Computer Applications*, vol. 41, pp. 424–440, 2014.

[4] S. Bhardwaj, L. Jain, and S. Jain, "Cloud computing: A study of infrastructure as a service (iaas)," *International Journal of engineering and information Technology*, vol. 2, no. 1, pp. 60–63, 2010.

[5] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, "Elasticity in cloud computing: state of art and research challenges," *IEEE Transactions on Service Computing*, vol. 11, no. 2, pp. 430–447, 2017.

[6] C. Pahl, "Containerization and the paas cloud," *IEEE Cloud Computing*, vol. 2, no. 3, pp. 24–31, 2015.

[7] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proceedings of USENIX Annul Technical Conference*, 2020.

[8] M. Bilal, M. Canini, R. Fonseca, and R. Rodrigues, "With great freedom comes great opportunity: Rethinking resource allocation for serverless functions," in *Proceedings of European Conference on Computer Systems*, 2023, pp. 381–397.

[9] A. Mahgoub, E. B. Yi, K. Shankar, S. Elnikety, S. Chaterji, and S. Bagchi, "Orion and the three rights: Sizing, bundling, and prewarming for serverless dags," in *Proceedings of Symposium on Network System Design and Implementation*, 2022.

[10] A. Suresh, G. Somashekar, A. Varadarajan, V. R. Kakarla, H. Upadhyay, and A. Gandhi, "Ensure: Efficient scheduling and autonomous resource management in serverless environments," in *IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*. IEEE, 2020.

[11] H. Qiu, W. Mao, A. Patke, C. Wang, H. Franke, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, "Reinforcement learning for resource management in multi-tenant serverless platforms," in *Proceedings of European Workshop on Machine Learning and Systems*, 2022.

[12] N. Akhtar, A. Raza, V. Ishakian, and I. Matta, "Cose: Configuring serverless functions using statistical learning," in *IEEE International Conference on Computer Communications*. IEEE, 2020, pp. 129–138.

[13] Z. Wen, Y. Wang, and F. Liu, "Stepconf: Slo-aware dynamic resource configuration for serverless function workflows," in *IEEE International Conference on Computer Communications*. IEEE, 2022, pp. 1868–1877.

[14] T. Zubko, A. Jindal, M. Chadha, and M. Gerndt, "Maff: Self-adaptive memory optimization for serverless functions," in *European Conference on Service-Oriented and Cloud Computing*. Springer, 2022, pp. 137–154.

[15] S. Eismann, L. Bui, J. Grohmann, C. Abad, N. Herbst, and S. Kounev, "Sizeless: Predicting the optimal size of serverless functions," in *Proceedings of International Middleware Conference*, 2021.

[16] G. Yu, P. Chen, Z. Zheng, J. Zhang, X. Li, and Z. He, "Faasdeliver: Cost-efficient and qos-aware function delivery in computing continuum," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3332–3347, 2023.

[17] Z. Li, Y. Liu, L. Guo, Q. Chen, J. Cheng, W. Zheng, and M. Guo, "Faasflow: Enable efficient workflow execution for function-as-a-service," in *Proceedings of International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 782–796.

[18] Z. Li, C. Xu, Q. Chen, J. Zhao, C. Chen, and M. Guo, "Dataflower: Exploiting the data-flow paradigm for serverless workflow orchestration," in *Proceedings of International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023, pp. 57–72.

[19] W. Wang, Q. Wu, Z. Zhang, J. Zeng, X. Zhang, and M. Zhou, "A probabilistic modeling and evolutionary optimization approach for serverless workflow configuration," *Software: Practice and Experience*, vol. 54, no. 9, pp. 1697–1713, 2024.

[20] G. Safaryan, A. Jindal, M. Chadha, and M. Gerndt, "Slam: Slo-aware memory optimization for serverless applications," in *IEEE International Conference on Cloud Computing*. IEEE, 2022, pp. 30–39.

[21] A. Raza, N. Akhtar, V. Isahagian, I. Matta, and L. Huang, "Configuration and placement of serverless applications using statistical learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1065–1077, 2023.